

AWS Cloud Architect — Technical Interview Prep

Principal Cloud Engineer level · Q&A knowledge base

Focus areas: AWS core & architecture · IaC · Containers & orchestration · CI/CD · Observability · Security · Networking · GenAI infra

Each question below is the kind of **technical challenge question** an interviewer uses to verify depth, not to ask about your résumé. Answers are compressed to the core reasoning and trade-offs you'd say out loud.

— Orange bar = high-probability / core question for this role.

AWS Core & Architecture

compute · storage · databases · Well-Architected

Q How do you design a highly available, multi-AZ web application on AWS?

Spread across ≥ 2 AZs behind an **ALB**; stateless app tier in an **Auto Scaling Group**; managed data tier with **RDS Multi-AZ** (synchronous standby) or Aurora. Push static assets to **S3 + CloudFront**. Store session/state externally (ElastiCache, DynamoDB) so any instance is disposable. Health checks + ASG replace failed nodes automatically. AZ failure is survived by design; Region failure needs cross-Region replication + Route 53 failover.

Q Walk through the 6 pillars of the Well-Architected Framework and a tension between two of them.

Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability. Classic tension: **Reliability vs Cost** — Multi-AZ RDS doubles standby cost; cross-Region DR multiplies it. You resolve it with the business RTO/RPO, not with a default. Another: **Performance vs Cost** — provisioned capacity vs on-demand/serverless.

Q When would you choose ECS Fargate vs EKS vs Lambda for a workload?

Lambda: event-driven, spiky, short (<15 min) tasks, want zero infra ops. **Fargate**: containerized long-running services, want serverless containers without node management. **EKS**: need Kubernetes ecosystem (operators, Helm, portability, complex scheduling) or multi-cloud strategy — accept the operational overhead. Decision driver is control vs operational burden, plus whether the team already speaks Kubernetes.

Q Difference between EBS, EFS, and S3 — and when each fits.

EBS: block storage, single-instance (or Multi-Attach io2), like a virtual disk — databases, boot volumes. **EFS**: NFS shared file system, multiple instances mount concurrently, scales elastically. **S3**: object storage over HTTP, effectively infinite, 11 nines durability — assets, backups, data lakes, logs. Not a filesystem; no in-place edits.

Q How do you pick between RDS, Aurora, and DynamoDB?

RDS: standard relational, known engines, moderate scale. **Aurora**: relational but need higher throughput, faster failover, storage auto-scaling to 128 TB, read replicas up to 15. **DynamoDB**: key-value/document at massive scale, single-digit-ms latency, serverless — but you must design around access patterns, not normalized tables. Relational joins/ad-hoc queries push you back to RDS/Aurora.

Q S3 storage classes and lifecycle strategy for cost?

Standard → Standard-IA (infrequent) → Glacier Instant/Flexible → Deep Archive, by access frequency. Use **lifecycle rules** to transition automatically by object age, and **Intelligent-Tiering** when access patterns are unknown. Watch retrieval fees and minimum storage durations on cold tiers.

Networking & VPC

subnets · routing · connectivity · DNS

Q Design a secure VPC layout for a 3-tier app.

Public subnets: ALB + NAT Gateway only. **Private subnets**: app tier, no public IPs, egress via NAT. **Isolated/private subnets**: database tier, no internet route at all. Security groups per tier, referencing each other by SG ID (not CIDR) so only the ALB SG can reach the app SG, only the app SG can reach the DB SG. NACLs as a coarse stateless backstop. Spread subnets across AZs.

Q Security Group vs NACL — key differences?

Security Group: stateful (return traffic auto-allowed), instance/ENI level, allow-rules only. **NACL:** stateless (must allow both directions), subnet level, supports explicit deny, evaluated in rule-number order. SGs are your primary control; NACLs handle subnet-wide blanket blocks.

Q How does traffic reach an instance in a private subnet, and how does that instance reach the internet?

Inbound: only via something in a public subnet (ALB, bastion) — there's no direct route from IGW. Outbound: private subnet route table sends `0.0.0.0/0` to a **NAT Gateway** in a public subnet, which has the actual IGW route. For AWS API calls, prefer **VPC Endpoints** (Gateway for S3/DynamoDB, Interface/PrivateLink for others) to skip NAT entirely — cheaper and keeps traffic off the public internet.

Q Options for connecting on-prem to AWS, and the trade-off.

Site-to-Site VPN: quick, cheap, over public internet, encrypted, variable latency. **Direct Connect:** dedicated private link, consistent low latency and bandwidth, but lead time and cost. Common pattern: Direct Connect with a VPN backup for resilience. For many VPCs, terminate on a **Transit Gateway** hub.

Infrastructure as Code

Terraform · CloudFormation · state · modules

Q How do you manage Terraform state safely across a team?

Remote backend — **S3 for state + DynamoDB for state locking** (or the newer S3-native locking). Enable versioning + encryption on the bucket. Never commit state to git (it holds secrets). Isolate state per environment/component so a `dev` apply can't touch `prod`. Use `terraform plan` in CI as a mandatory review gate.

Q Terraform vs CloudFormation vs Pulumi — how do you choose?

Terraform: multi-cloud, huge provider ecosystem, HCL, mature modules — the default for agency/multi-client work. **CloudFormation:** AWS-native, no state file to manage (AWS tracks it), tight service integration, drift detection built in. **Pulumi:** real languages (TS/Python/Go) — good when teams want loops/abstractions and to reuse app-language tooling. For a consultancy serving varied clients, Terraform's portability usually wins.

Q What makes a good reusable Terraform module?

Single clear responsibility, sensible defaults, well-typed variables with validation, meaningful outputs, no hardcoded account/region/env values, and pinned provider versions. Version the module (registry or git tag) so consumers upgrade deliberately. Avoid over-parameterizing — a module that exposes 80 variables is just Terraform with extra steps.

Q How do you handle secrets in IaC pipelines?

Never in code or state as plaintext. Pull at runtime from **Secrets Manager** or **SSM Parameter Store (SecureString)**; inject via CI variables or OIDC-assumed roles. Terraform still writes referenced values into state, so treat state as sensitive (encrypted, access-controlled) and prefer having the *application* fetch the secret directly rather than passing it through Terraform.

Q How do you detect and remediate configuration drift?

`terraform plan` (or CloudFormation drift detection) on a schedule flags out-of-band changes. Remediate by re-applying IaC or importing the change back into config. Best long-term fix is prevention: restrict console write access in prod so all change flows through the pipeline.

Containers & Orchestration

Docker · Kubernetes · ECS/EKS · scaling

Q Explain the core Kubernetes objects and how a request reaches a pod.

Pod (smallest deployable unit) is managed by a **Deployment** (via ReplicaSet) for rollout/scaling. A **Service** gives a stable virtual IP/DNS and load-balances across pods by label selector. External traffic enters through an **Ingress** (+ controller, e.g. ALB Controller) → Service → kube-proxy → pod. **ConfigMaps/Secrets** supply config; **namespaces** isolate tenants.

Q How does autoscaling work in Kubernetes on EKS?

Two layers: **HPA** scales pod replicas on CPU/memory/custom metrics; **Cluster Autoscaler** (or **Karpenter**) adds/removes nodes when pods can't be scheduled. Karpenter is now preferred on EKS — it provisions right-sized nodes fast and bin-packs for cost. VPA can adjust pod requests but conflicts with HPA on the same metric.

Q How do you make a Docker image production-ready?

Multi-stage builds to keep the runtime image tiny; minimal/distroless base; run as **non-root**; pin versions; `.dockerignore` to shrink context and avoid leaking secrets; one process per container; declare health checks. Scan the image (Trivy/ECR scanning) in CI before push.

Q ECS vs EKS operationally?

ECS: AWS-proprietary, far simpler, deep AWS integration, less to operate — great when you don't need Kubernetes. **EKS**: standard Kubernetes API, portable, rich ecosystem, but you own upgrades, add-ons, and more moving parts. Pick ECS for speed and lower ops; EKS for portability and ecosystem needs.

Q How do rolling deployments and health checks prevent downtime?

New pods/tasks start and must pass **readiness** checks before receiving traffic; old ones drain only after new ones are healthy (maxUnavailable/maxSurge control the pace). Failed readiness halts the rollout instead of serving errors. Pair with **liveness** checks to restart hung containers. Enables blue/green or canary via traffic shifting.

CI/CD & Deployment

GitLab CI · GitHub Actions · pipelines · strategies

Q Design a CI/CD pipeline from commit to production.

Commit → **lint + unit tests** → build artifact/image → **scan** (SAST, image, IaC) → push to registry → deploy to staging → integration/smoke tests → **manual approval gate** → deploy to prod (rolling/canary) → post-deploy verification + auto-rollback on failure. Keep it fast for feedback; fail early on the cheap checks.

Q Blue/green vs canary vs rolling — trade-offs?

Rolling: gradual replacement, cheap, but mixed versions briefly serve traffic. **Blue/green**: full parallel environment, instant cutover + instant rollback, but ~2× infra during switch. **Canary**: send a small % to the new version, watch metrics, then ramp — safest for risk, needs good observability and traffic-splitting. Choose by blast-radius tolerance and cost.

Q How do you authenticate a CI pipeline to AWS without long-lived keys?

OIDC federation: GitLab/GitHub acts as an identity provider; the pipeline assumes an IAM role via short-lived STS credentials scoped by trust policy (e.g. specific repo/branch). No static access keys to rotate or leak. This is the modern best-practice answer and a strong signal in an interview.

Q How do you keep a monorepo's pipeline efficient?

Change detection so only affected services build/deploy; caching (dependencies, layers, Terraform providers); parallel jobs; reusable pipeline templates/components. Fail fast by ordering cheap checks first.

Observability & Operations

metrics · logs · traces · alerting · incidents

Q What are the three pillars of observability and how do they fit together?

Metrics (numeric time-series — what's wrong, cheap to store), **logs** (discrete events — details/context), **traces** (request path across services — where latency/failure occurs). Flow: a metric alert fires → traces localize the failing hop → logs explain the root cause. On AWS: CloudWatch metrics/logs, X-Ray or OpenTelemetry for tracing; Prometheus/Grafana/ELK as an alternative stack.

Q How do you set alerts that are actionable, not noisy?

Alert on **symptoms/SLOs** (error rate, latency, saturation) not raw causes; page only on user-impacting conditions. Use the **four golden signals** (latency, traffic, errors, saturation). Add thresholds with duration windows to avoid flapping, severity tiers (page vs ticket), and runbook links on each alert. Kill alerts nobody acts on.

Q Define SLI, SLO, SLA and how error budgets drive decisions.

SLI: measured indicator (e.g. % successful requests). **SLO**: internal target for that SLI (e.g. 99.9%). **SLA**: external contractual promise (usually looser, with penalties). The gap below 100% is the **error budget**: if you're burning it fast, freeze risky releases and prioritize reliability; if it's healthy, ship faster.

Q Walk through your incident response process.

Detect (alert/monitor) → declare severity → assign an **incident commander** → mitigate first (roll back, scale, failover) before root-causing → communicate status to stakeholders → resolve → **blameless post-mortem** with timeline and action items. Fix the systemic cause and the detection gap, not just the symptom.

Security

IAM · encryption · secrets · least privilege

Q Explain how an IAM policy is evaluated (roles, policies, precedence).

Default **deny**. A request is allowed only if an applicable policy explicitly allows it *and* no policy explicitly denies it — an **explicit deny always wins**. Effective permissions are the intersection of identity policies, resource policies, permission boundaries, and SCPs (Organizations). Prefer **roles** (temporary STS credentials) over IAM users with static keys.

Q How do you implement least privilege in practice?

Start from zero and add only needed actions/resources; scope by resource ARN and conditions (e.g. tag, source VPC, MFA). Use **permission boundaries** to cap what teams can grant, **Access Analyzer** to find over-broad or external access, and review CloudTrail/last-accessed data to prune unused permissions. Avoid ***** in production policies.

Q Encryption at rest vs in transit on AWS, and KMS's role.

In transit: TLS everywhere (ALB/CloudFront certs via ACM). **At rest**: enable SSE on S3/EBS/RDS backed by **KMS** keys. KMS manages key lifecycle, rotation, and grants; use **customer-managed keys** when you need control over policy/rotation/audit, and key policies + grants to gate who can decrypt. Envelope encryption keeps data keys wrapped by the CMK.

Q Secrets Manager vs SSM Parameter Store?

Secrets Manager: built for secrets — automatic rotation, cross-Region replication, higher cost. **Parameter Store**: config + secrets (SecureString), cheaper/free tier, no native rotation. Use Secrets Manager when you need rotation (DB creds); Parameter Store for general config and simpler secrets.

Q How do you secure a multi-account AWS setup?

AWS Organizations with **SCPs** as guardrails (deny regions, deny disabling CloudTrail, etc.), separate accounts per environment/workload for blast-radius isolation, centralized logging + security tooling in dedicated accounts, and **Control Tower / Landing Zone** to provision accounts consistently. Cross-account access via assumed roles, never shared credentials.

AI / GenAI Infrastructure

Bedrock · RAG · vector DBs · inference

Q Explain the architecture of a RAG system on AWS.

Ingest: documents → chunk → **embed** (Bedrock/Titan or similar) → store vectors in a **vector DB** (OpenSearch k-NN, Aurora pgvector, or a managed store). Query: embed the user question → similarity search for top-k chunks → inject them as context into the prompt → LLM (via **Bedrock**) generates a grounded answer. Adds freshness and source-grounding without retraining, and lets you cite sources.

Q Why use a vector database, and what are the trade-offs in choosing one?

It does fast **approximate nearest-neighbor** search over high-dimensional embeddings — infeasible with normal indexes. Trade-offs: recall vs latency (ANN algorithm/params like HNSW), managed vs self-hosted ops, cost at scale, and whether you can reuse existing infra (pgvector on Aurora) vs a dedicated engine (OpenSearch). Pick based on scale, existing stack, and ops appetite.

Q How do you make LLM inference scalable and cost-controlled?

Prefer managed **Bedrock** to avoid GPU ops; use on-demand for spiky load and provisioned throughput for steady high volume. Control cost with **caching** (semantic/response cache), prompt/context trimming, right-sizing the model to the task (small model for easy calls), token limits, and async batching. Monitor per-request token cost as a first-class metric.

Q What are the main risks when putting a GenAI system into production?

Prompt injection and data exfiltration, hallucination (mitigate with RAG grounding + guardrails), sensitive data leaking into prompts/logs, cost blow-ups from unbounded usage, and latency. Add input/output guardrails, PII redaction, rate limiting, and evaluation/monitoring of answer quality — treat it as an untrusted-input system.

Q A client's site slows down under traffic spikes. How do you diagnose and fix it?

Diagnose with the **golden signals**: check where latency/saturation appears (LB, app, DB). Common fixes: horizontal **auto-scaling** on the app tier, **CloudFront + caching** for static/edge content, **read replicas** or connection pooling for DB pressure, and an async queue (SQS) to absorb bursts. Load-test to validate. Distinguish a scaling problem from an inefficiency (N+1 queries) before throwing hardware at it.

Q How would you plan a migration of a legacy on-prem app to AWS?

Assess and pick a **7 Rs** strategy per component (rehost, replatform, refactor...). Start with a low-risk pilot, set up landing zone/networking (VPC, Direct Connect/VPN), migrate data (DMS) with a cutover plan and rollback, run in parallel to validate, then switch DNS. Prioritize by business value and risk; don't refactor everything on day one.

Q How do you approach disaster recovery and choose an RTO/RPO strategy?

Four patterns by cost/speed: **Backup & Restore** (cheapest, slow), **Pilot Light**, **Warm Standby**, **Multi-Site Active/Active** (fastest, most expensive). Let the business **RTO** (how fast to recover) and **RPO** (acceptable data loss) pick the pattern — over-engineering DR wastes money, under-engineering risks the business. Test failover regularly; an untested DR plan doesn't exist.

Q A client wants to cut their AWS bill 30%. Where do you look?

Right-size over-provisioned instances/volumes; buy **Savings Plans / Reserved Instances** for steady baseline load; use **Spot** for fault-tolerant/batch work; S3 lifecycle to colder tiers; delete orphaned resources (unattached EBS, idle NAT/ELB, old snapshots); consolidate NAT Gateways and use VPC endpoints; enable Cost Explorer/Compute Optimizer recommendations. Tag for cost allocation so you can see where money actually goes.